Opponent-Modulated Value Networks for Exploiting Suboptimal Play in Leduc Hold'em

Chris He **Devang Pant** **Rutvij Kharat** **Mark Dang**
University of California, San Diego

Abstract

Counterfactual Regret Minimization (CFR)-based methods have achieved superhuman performance in poker by approximating Nash Equilibrium strategies. Despite strong performance against professional players, these models assume the opponent to be near-optimal, limiting their ability to extract value from exploitable, suboptimal opponents. In this work, we propose a method that adapts to opponent playing styles on the fly, consisting of a pre-trained base value network and a value-modulation head for online opponent modeling. Evaluated in a round-robin tournament across diverse opponent styles in Leduc Hold'em, our method achieves +21.1 more chips per session compared to CFR, demonstrating better exploitation of suboptimal play. Furthermore, while baseline methods' performance degrades on unseen opponent styles, our method maintains consistent performance gains across seen and unseen playing styles.

1 Introduction

Recent advancement in poker AI has demonstrated superhuman performance across various versions of Texas Hold'em, including Heads-Up Limit [2], Heads-Up No-Limit [3, 4, 7], and Six-Player No-Limit [5]. These methods predominantly leverage Counterfactual Regret Minimization (CFR) techniques to approximate and play a Nash Equilibrium strategy during test-time. In other words, the policy computed implicitly assumes that opponents play near-optimally, which is valid in matchups against professional poker players.

However, in more casual and realistic settings of gameplay, opponents are not near-optimal. Instead, they exhibit exploitable tendencies, such as over-folding or excessive aggression. When played against suboptimal opponents, CFR-based approaches achieve non-negative expected value (i.e., guaranteed bounded loss), yet they cannot fully capitalize on exploitable patterns. This leads to the central question of this work: To what extent does the equilibrium assumption limit performance against suboptimal, real-world opponents, and how do opponent-aware strategies address this limitation?

We introduce an opponent-modulated value network, utilizing a pre-trained base value network and a value-modulation head for online opponent modeling. During multiple rounds of gameplay against the same opponent, our agent collects opponent statistics, and leverages it via the value-modulation head to inform better decision-making. We evaluated our network against CFR-based agents in a round-robin tournament with a diverse pool of opponents and playing styles, demonstrating that adaptive exploitation yields higher average chip gain per round than equilibrium-seeking strategies.

2 Background

2.1 Information Sets

A core characteristic of imperfect information games is that the full game state isn't observable by players. In poker, the complete state of a game consists of players' private cards, a public card (if

dealt), pot size, etc. However, players can only observe their own private hand. In other words, the same game state that a player observes (e.g., private card is K and public card is J) can correspond to multiple actual game states (e.g., opponent can be holding J, Q, or K). This set of game states that a player, given their limited information, cannot distinguish between is defined as an information set. In principle, a player's policy is a function of the information set they are in, not the actual game state, because the latter is unknown and unobservable.

2.2 Counterfactual Regret Minimization (CFR)

CFR is an iterative algorithm for approximating the Nash Equilibrium strategy of an incomplete information game. At every information set of a player, the regret of each available action (under the current iteration's strategy profile) is determined. The accumulated regret over existing iterations is used to construct next iteration's strategy profile, and the average strategy profile over sufficient iterations is theoretically guaranteed to converge to a Nash Equilibrium strategy profile.

CFR cannot be directly applied to poker, because it is infeasible to exactly compute the $\sim 10^{14}$ information sets. Libratus and Pluribus utilize card and action abstraction, grouping similar information sets and actions together to reduce computation, and apply Monte Carlo CFR [6] on the abstracted version of the game. DeepStack and ReBel uses a value function to approximate counterfactual values at a leaf node, limiting the search depth and computation by avoiding to search until a terminal node is reached.

However, CFR assumes all players follow the same iterative procedure when acting, and it outputs a strategy that is optimal only when opponents are also playing Nash Equilibrium strategies. In other words, existing CFR-based methods assume optimal gameplay from the opponents, and cannot utilize or exploit opponent behavior that deviates from optimal gameplay.

2.3 Leduc Hold'em

In this work, we studied a variant of poker widely used in poker game-theory research, called Leduc Hold'em. Leduc Hold'em consists of a 6-card deck (2 Jacks, 2 Queens, 2 Kings), 2 betting rounds (preflop and flop), 2 players, and 3 possible actions (call, raise, and fold) at each decision point. A detailed walkthrough of Leduc Hold'em rules and gameplay is included in Appendix A.

Leduc Hold'em preserves the core of poker — imperfect information (both players are dealt private hands that the opponent cannot observe) and strategic complexity (behavior like bluffing). Hence, results from this work can be extended to or adapted in other poker variants, such as Texas Hold'em. Furthermore, Leduc Hold'em drastically reduces computation complexity (only 288 information sets [1]), allowing us to focus on algorithmic innovation without large training overhead. In addition, this means CFR can be used to solve this game exactly, without abstraction, providing a valuable baseline for us to evaluate our result.

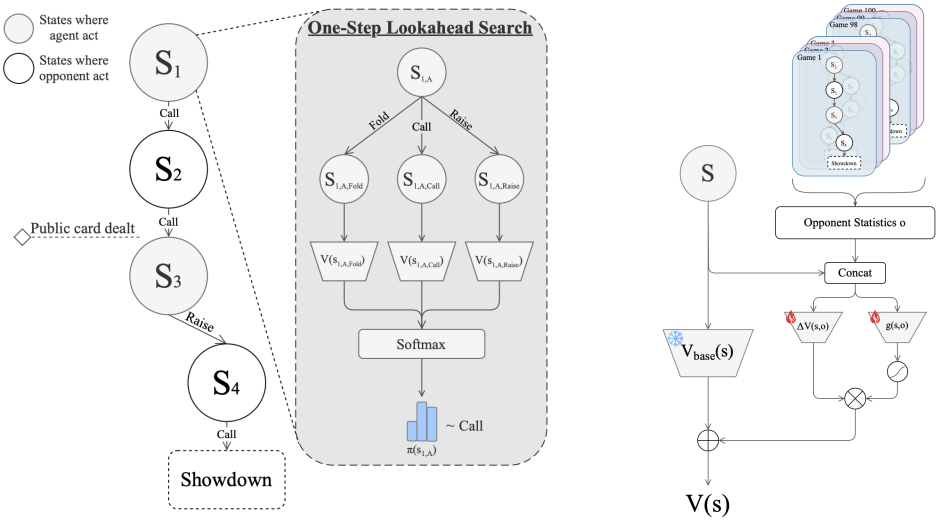
3 Methodology

Our proposed method consists of 1) a base value network $V_{\text{base}}(s)$ that approximates the expected value of an information set (assuming optimal gameplay onwards), and 2) a value modulation head $\Delta V(s, o)$ that uses collected opponent statistics o to adjust base model's value estimate, optionally through a gating mechanism $g(s, o)$.

3.1 Base Value Network Description

Architecture. The base value network's backbone is a Multilayer Perceptron (MLP) with 2 hidden layers of 64 dimensions. We experimented with larger networks, but the increased capacity yielded marginal return. $V_{\text{base}}(s)$ receives a state representation $s \in \mathbb{R}^{15}$ as input, and outputs a scalar that approximates the expected terminal reward of the current player, assuming both players follow a Nash Equilibrium strategy profile from state s onwards. We presented a table illustrating the elements included in our state representation in Appendix B.

Self-play. We trained the base value network using self-play with TD(0) loss. During self-play, each agent selects its action via one-step lookahead search (Fig. 1(A)). For example, at the first time



(A) One-step lookahead search **(B)** Value network architecture

Figure 1: (A) One-step lookahead search for inference-time action selection. (B) Opponent-modulated value network with pre-trained base and value modulation head.

step where agent A acts, it looks at the successor states (i.e., $s_{1,A,\text{call}}$ and $s_{1,A,\text{raise}}$) that it can reach through a legal action $a_{\text{legal}} \in \mathcal{A}_A$. Each state is evaluated using the value function $V(\cdot)$, and the value estimates are passed into a softmax operator to construct a mixed strategy from which an action a_{chosen} is sampled. Formally, agent A selects its action a at its decision point $s_{t,A}$ via

$$\pi(a \mid s_{t,A}) = \frac{\exp(V(s_{t,A,a})\,/\tau)}{\sum_{a_{\text{legal}} \in \mathcal{A}_A} \exp\big(V\big(s_{t,A,a_{\text{legal}}}\big)\,/\tau\big)} \tag{1}$$

where $a \in \mathcal{A}_A$ and τ is a temperature parameter.

We set $\tau = 1$ in all experiments. Greedy-selection isn't used in inference-time since deterministic strategies are easily exploitable in poker.

Notably, the one-step lookahead formulation we presented is equivalent to learning a Q-network, but it has the additional flexibility to handle continuous action spaces (e.g., arbitrary bet sizes) since the state after an action can always be computed (the state transition function is known). This offers value network formulations better generalization to other poker variants.

Temporal Difference (TD) loss and training. We used temporal difference loss as the objective function. Specifically, we matched the value estimate of a visited state $s_{t,A,a_{\text{chosen}}}$ to the value of the next visited state $s_{t+2,A,a_{\text{chosen}}}$ ($t{+}2$ since turns alternate and $t{+}1$ is when opponent acts):

$$\mathcal{L}_{\text{TD}(0)} = \mathbb{E}\Big[\big(V(s_{t,A,a_{\text{chosen}}}) - \text{sg}(V(s_{t+2,A,a_{\text{chosen}}}))\big)^2\Big] \tag{2}$$

where $\text{sg}(\cdot)$ denotes stop-gradient since we treat it as the bootstrap target.

The value of the state immediately before terminal state is matched to the terminal reward (i.e., chips earned that round). We also experimented with TD(1) and TD(2), but observed highly unstable training. We found that higher order TD targets frequently reduce to the trajectory's terminal reward (high variance) at later time steps, due to Leduc Hold'em's relatively short time horizon (at most 8 actions before reaching a terminal state). We detailed our training hyperparameters in Appendix C.

3.2 Value Modulation Head

The value modulation head $\Delta V(s, o)$ adjusts the base value model $V_{\text{base}}(s)$'s estimate using observed opponent statistics o, producing a composite value estimate:

$$V(s) = V_{\text{base}}(s) + \Delta V(s, o) \cdot g(s, o) \tag{3}$$

which approximates state s's expected value, assuming opponent plays a strategy encoded in opponent statistics o. In this work, we uses hand-coded features for o rather than training an end-to-end opponent encoder. This potentially simplified the modulation head's task to learning different value adjustments for different playing styles summarized in o.

Opponent statistics. We introduced the notion of sessions, where each session consists of $n = 100$ consecutive games played between two agents. As a session progresses, each agent gradually accumulate statistics about the opponent (e.g., preflop fold rate and re-raise rate; details presented in Appendix D). We also included a scalar $\rho = \frac{t}{n} \in [0, 1]$, representing the reliability of the collected statistics, where t is the number of observed rounds. To mitigate the cold start problem, we utilized a prior (average statistics of agents in training pool) via Beta-Bernoulli smoothing, with prior strength $S = 20$.

Architecture and gating mechanism. We used a MLP with two 32-dimension hidden layers for the value modulation head. We included an optional gate, conditioned on opponent statistics and the current game state, to control the influence of the modulation. This is motivated by the observation that some states require little modulation (different opponents generate the same EV), while in others, the state's EV varies greatly across opponents.

Pool training. We trained the value modulation head with a similar recipe as the base value network. Instead of self-play, our agent plays against an opponent sampled from the training pool for each session. The training pool contains 6 hand-crafted rule-based agents representing distinct playing styles, a CFR-based agent, and the base value model (see Appendix E for composition details). This setup exposes our agent to a wide range of playing styles, allowing it to better learn how different opponent statistics can influence value. In addition, we freeze the base value network's weights during training because it is not conditioned on opponent statistics (it will just overfit the training pool if allowed to update).

4 Results

Table 1: Agent performance in chips per session. Robustness **Rob.** $= \mu - 1.5\sigma$ (higher is better).

	In-Distribution		OOD		Overall	
Agent	Avg	Rob.	Avg	Rob.	Avg	Rob.
CFR (Nash)	$+25.6$	-30.8	$+12.3$	-16.4	$+20.8$	-28.4
Value Network	$+44.7$	-29.5	-25.9	-25.9	$+35.9$	-41.8
Modulated Value Net. (Ours)	$+50.1$	-31.1	$\mathbf{+27.5}$	-20.7	$\mathbf{+41.9}$	-31.0
REINFORCE	$+53.4$	-28.3	$+12.8$	$\mathbf{-2.1}$	$+38.6$	-33.4
Actor-Critic	$+2.2$	-94.3	-27.7	-59.0	-8.7	-90.8
DQN	$\mathbf{+59.1}$	$\mathbf{-5.4}$	$+5.4$	-23.1	$+39.6$	$\mathbf{-27.1}$

We evaluated our method against baseline models including CFR, Policy Gradient (REINFORCE), Actor-Critic, and Double Q-Learning (DQN). These models are trained with similar budgets against the same training pool used for the value modulation head. To evaluate each model, we played it against all agents in the training (In-Distribution) and testing (Out-Of-Distribution, OOD) pool, 10000 games (100 sessions) per opponent. In Figure 2, we reported each model's performance relative to a CFR baseline. For each opponent (x-axis), we measured the chips/session earned by each model, and subtract from that the chips/session earned by the CFR-based model. This measures the additional value each model extracts by deviating from optimal play.

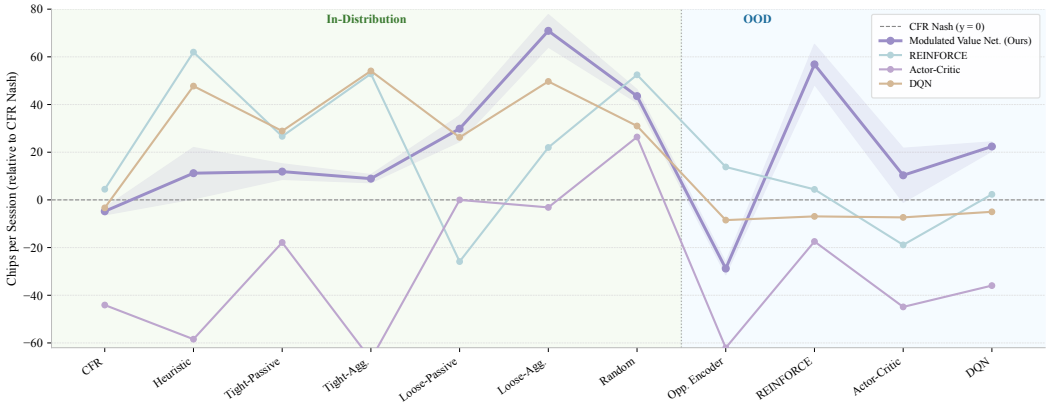


Figure 2: Model performance against opponents, relative to CFR-based model.

Our method (bolded purple, error bands showing mean ± 1 standard deviation over 3 random seeds) demonstrated consistent performance gains over CFR-based model, maintaining positive net-gain against almost all opponents. Although baseline methods exhibit better exploitation of rule-based agents (in-distribution), the policy they learned are not generalized, and their performance collapsed against out-of-distribution (OOD) agents. Quantitatively, our method demonstrated best performance overall (+41.9 chips per session), and over-performs baseline methods by a significant margin for OOD agents (Table 1).

4.1 Value Modulation Head Analysis

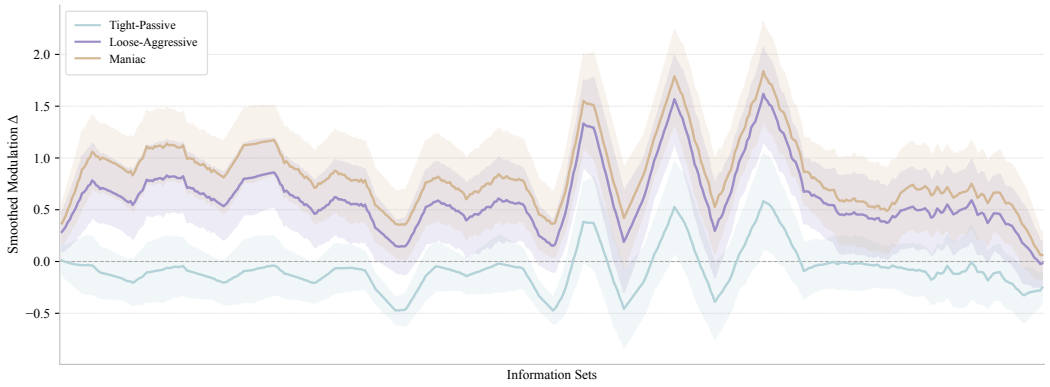


Figure 3: Value modulation head output across information sets for representative opponent types.

We plotted the value modulation head's output (y-axis) at every possible information set in Leduc Hold'em (x-axis), for three of the rule-based agents with distinct styles. The modulation head exhibits differentiation across opponent types (e.g., positive modulation against loose-aggressive opponent (purple) who raise too often, versus negative modulation against tight-passive (tint)). However, it demonstrated limited capability for fine-grained modulation between states, hinting that it mainly learned a useful global adjustment to all states for different opponent types.

4.2 Ablation Study

To understand the contribution of each component of our method, we conducted an ablation study summarized in Table 2.

Table 2: Ablation study on opponent-modulated value network.

	In-Distribution		OOD		Overall	
Method	Avg	Rob.	Avg	Rob.	Avg	Rob.
Value Net. (self-play)	$+44.7$	-29.5	-25.9	-25.9	$+35.9$	-41.8
+ State Mod. (no opp. stats)	$+51.8$	-24.9	-28.0	-28.0	$+41.8$	-40.1
+ Opp. Stats Mod., frozen base (Ours)	$+50.1$	-31.1	$\mathbf{+27.5}$	-20.7	$+41.9$	-31.0
+ Opp. Stats Mod., unfrozen base	$\mathbf{+62.7}$	$\mathbf{-10.1}$	-17.6	$\mathbf{-17.6}$	$\mathbf{+52.6}$	$\mathbf{-26.2}$
Joint Training from scratch	$+35.9$	-30.6	-16.6	-16.6	$+29.4$	-38.1

Impact of opponent statistics. Compared to the base value network, adding modulation using opponent statistics yielded higher average return. To account for the increased capacity brought by the modulation head (a 3 layer MLP), we trained another modulation head that does not use opponent statistics as input (+ State Mod., no opp. stats). This produced a modest in-distribution gain but caused a significant drop in OOD performance. This confirmed that value modulation head's pool-training does allow it to learn opponent-value modulation mappings that are generalizable to opponents of unseen playing style.

Impact of freezing the base network. Training with an unfrozen base network (+ Opp. Stats Mod., unfrozen base) achieves the highest in-distribution average (+62.7), but collapses on OOD opponents (-17.6). This supports our hypothesis that an unfrozen base network absorbs pool-specific patterns into its weights, sacrificing generality for in-distribution fit.

Joint training from scratch. Training the full composite network end-to-end from scratch performs worst overall (+29.4 avg, -38.1 robustness), suggesting that a well-initialized base network is critical — the modulation head alone cannot compensate for a poorly learned value prior.

5 Conclusion

We proposed an opponent-modulated value network for exploiting suboptimal play in Leduc Hold'em, combining a pre-trained base value network with a value modulation head that conditions on online-collected opponent statistics to adjust value estimates at inference time. This addresses a known limitation of equilibrium-based approaches, which assume near-optimal opponents and cannot adapt to exploitable playing styles.

In a round-robin tournament across diverse opponent styles, our method achieves $+41.9$ chips per session overall, outperforming CFR by $+21.1$ chips per session, and maintains positive net gain on out-of-distribution opponents where non-adaptive baselines degrade. Ablation results suggest that the improvement is attributable to the opponent statistics signal rather than increased model capacity alone, and that freezing the base network weights is important for out-of-distribution generalisation.

Several limitations remain for future work. First, the opponent statistics used are hand-coded aggregate features; replacing these with a learned opponent encoder could capture higher-order behavioural patterns that the current representation may miss. Second, the modulation head appears to learn primarily global per-opponent adjustments, with limited state-level granularity, suggesting that closer integration with the lookahead search may yield further gains. Third, the current evaluation is limited to Leduc Hold'em, and extending the approach to larger variants such as Texas Hold'em would help assess its scalability.

References

[1] (leduc hold'em intro) heads-up limit hold'em poker is solved – communications of the ACM. URL https://cacm.acm.org/research/heads-up-limit-holdem-poker-is-solved/.

[2] Michael Bowling, Neil Burch, Michael Johanson, and Oskari Tammelin. (cepheus) heads-up limit hold'em poker is solved. 60(11):81–88. ISSN 0001-0782. doi: 10.1145/3131284. URL https://dl.acm.org/doi/10.1145/3131284.

[3] Noam Brown and Tuomas Sandholm. Libratus: The superhuman AI for no-limit poker. In *Proceedings of the Twenty-Sixth International Joint Conference on Artificial Intelligence*, pages 5226–5228. International Joint Conferences on Artificial Intelligence Organization, . ISBN 978-0-9992411-0-3. doi: 10.24963/ijcai.2017/772. URL https://www.ijcai.org/proceedings/2017/772.

[4] Noam Brown and Tuomas Sandholm. (libratus) superhuman AI for heads-up no-limit poker: Libratus beats top professionals. 359(6374):418–424, . doi: 10.1126/science.aao1733. URL https://www.science.org/doi/full/10.1126/science.aao1733.

[5] Noam Brown and Tuomas Sandholm. (pluribus) superhuman AI for multiplayer poker. 365 (6456):885–890, . doi: 10.1126/science.aay2400. URL https://www.science.org/doi/10.1126/science.aay2400.

[6] Marc Lanctot, Kevin Waugh, Martin Zinkevich, and Michael Bowling. (MCCFR) monte carlo sampling for regret minimization in extensive games. In *Advances in Neural Information Processing Systems*, volume 22. Curran Associates, Inc. URL https://proceedings.neurips.cc/paper_files/paper/2009/hash/00411460f7c92d2124a67ea0f4cb5f85-Abstract.html.

[7] Enmin Zhao, Renye Yan, Jinqiu Li, Kai Li, and Junliang Xing. (AlphaHoldem) AlphaHoldem: High-performance artificial intelligence for heads-up no-limit poker via end-to-end reinforcement learning. 36(4):4689–4697. ISSN 2374-3468. doi: 10.1609/aaai.v36i4.20394. URL https://ojs.aaai.org/index.php/AAAI/article/view/20394.

A Leduc Hold'em Rules

Leduc Hold'em is a simplified two-player poker variant. The deck consists of six cards — two Jacks, two Queens, and two Kings — and at every decision point a player chooses from the action set $\mathcal{A} = \{\text{CALL}, \text{RAISE}, \text{FOLD}\}$.

The game has two betting rounds: *pre-flop* (before a community card is dealt) and *flop* (after the community card is revealed). A raise costs 2 chips in the pre-flop round and 4 chips in the flop round; at most two raises are allowed per round.

At the start of each hand, both players post a 1-chip ante (initial pot of 2 chips) and each receives one private card. Pre-flop betting begins; if neither player folds, a single public board card is then dealt and flop betting follows. The hand ends in one of two ways:

1. **Fold:** the opposing player wins the entire pot.
2. **Showdown:** both cards are revealed. A player whose private card matches the board card has a pair and wins. If neither player has a pair, the higher private card wins (K $>$ Q $>$ J).

Each agent observes only the information legally available to that player: their own private card, the (possibly hidden) board, the pot vector, the current round and acting player, the number of raises in the current round, and the public action history.

B State and Opponent-Statistics Representation

Game-state encoding. Every state s is encoded as a 15-dimensional vector $\phi(s) \in \mathbb{R}^{15}$ from the perspective of the *viewer* (the seat whose value is being estimated). The same encoding is used by both the base value network and the modulation head.

Table 3: 15-dimensional game-state encoding $\phi(s)$.

Component	Dims	Description
Private hand	3	One-hot over $\{\text{J}, \text{Q}, \text{K}\}$.
Board card	4	One-hot over $\{\text{J}, \text{Q}, \text{K}, \text{NONE}\}$.
Pot (viewer-relative)	2	$(\text{viewer chips}, \ \text{opp. chips})/13$.
My-turn flag	1	1 if it is the viewer's turn, else 0.
Position	1	Seat index of the viewer (0 or 1).
Current round	1	0 pre-flop, 1 flop.
Terminal flag	1	1 if the hand has ended, else 0.
Has-pair flag	1	1 if the private card matches the board.
Raises this round	1	Normalised by the per-round raise cap (2).

The pot normaliser 13 corresponds to the maximum total chip commitment attainable at showdown under the betting rules of Section A.

Modulation-head input. The value modulation head receives, in addition to $\phi(s)$, a 7-dimensional opponent-statistics vector o summarising the current opponent's observed behaviour over the ongoing session (Section D). The full modulation-head input is therefore $[\phi(s) \,\|\, o] \in \mathbb{R}^{22}$.

C Training Configuration

C.1 Base Value Network

The base value network $V_{\text{base}}(s)$ is a two-hidden-layer MLP $15 \to 64 \to 64 \to 1$ with ReLU activations, trained with self-play and a TD(0) objective. During training, action selection follows the Boltzmann distribution induced by the one-step lookahead values with temperature $\tau = 1.0$; at evaluation time, action selection is greedy. Hyperparameters are listed in Table 4.

Table 4: Training hyperparameters for the base value network.

Hyperparameter	**Value**
Episodes	200,000
Learning rate	1×10^{-4}
Optimiser	Adam (β_1=0.9, β_2=0.999)
Loss	MSE
Batch size	32 episodes
Architecture	$15 \to 64 \to 64 \to 1$ (ReLU)
Temperature τ	1.0

TD(0) target. For a per-player post-action state chain $(s_1, s_2, \ldots, s_T)$ with terminal reward r, the bootstrap target is

$$y(s_t) = \begin{cases} r & t = T, \\ \mathrm{sg}\big(V_{\text{base}}(s_{t+1})\big) & \text{otherwise}, \end{cases} \qquad \mathcal{L} = \mathrm{MSE}\big(V_{\text{base}}(s_t),\, y(s_t)\big),$$

where sg denotes the stop-gradient operator. Successive states in a chain are separated by two game time steps, since the opponent acts between consecutive learner actions.

C.2 Value Modulation Head

The modulation head implements $V(s, o) = V_{\text{base}}(s) + \Delta(s, o)$, where Δ is a small MLP mapping the concatenated 22-dimensional input through two hidden layers of 32 units to a scalar residual. The output-layer weights are initialised to $\mathcal{U}(-10^{-2},\, 10^{-2})$ and biases to zero, so that training begins close to the frozen base. Hyperparameters are listed in Table 5.

The modulation head is trained against the weighted opponent pool of Section E (CFR and the heuristic agent oversampled $3\times$) in sessions of $n = 100$ hands. The seat of the learner alternates between episodes (seat $=$ episode $\bmod 2$) so that both first- and second-acting decisions are observed. Gradient updates apply to the modulation parameters only, with V_{base} frozen at the self-play-trained checkpoint described above. We report mean and standard deviation across 3 random seeds.

Table 5: Training hyperparameters for the value modulation head (frozen-base variant; reported as "Modulated Value Net. (Ours)" in the main text).

Hyperparameter	**Value**
Episodes	300,000
Learning rate	1×10^{-4}
Optimiser	Adam
Loss	MSE on residual TD(0) targets
Modulation-head arch.	$22 \to 32 \to 32 \to 1$ (ReLU)
Output init.	$\mathcal{U}(-10^{-2}, 10^{-2})$, bias 0
Batch size	32 episodes
Session length n	100 hands
Prior strength S	20
Seeds	3

Residual TD(0) target. At each learner post-action state s_t with stats snapshot o_t:

$$y_\Delta(s_t, o_t) = \begin{cases} r - V_{\text{base}}(s_T) & t = T, \\ \big(V_{\text{base}}(s_{t+1}) + \Delta(s_{t+1}, o_{t+1})\big) - V_{\text{base}}(s_t) & \text{otherwise}, \end{cases}$$

$$\mathcal{L}_{\text{TD(0)}} = \mathrm{MSE}\big(\Delta(s_t, o_t),\, \mathrm{sg}(y_\Delta(s_t, o_t))\big).$$

The bootstrap target on non-terminal transitions is the *total* next-state value $V_{\text{base}} + \Delta$, so the residual head learns to be consistent with itself; only the modulation parameters receive gradient updates, while V_{base} remains frozen throughout.

Optional state-conditioned gate. The state-gated variant studied in our ablations augments the composite value as $V(s, o) = V_{\text{base}}(s) + g(s, o)\,\Delta(s, o)$, where $g : \mathbb{R}^{22} \to [0, 1]$ is a $22 \to 16 \to 16 \to 1$ MLP followed by a sigmoid and initialised with near-zero output weights. Conditioning the gate jointly on state and stats was motivated by the observation that opponent style has variable impact across states.

C.3 Baseline Reinforcement-Learning Agents

The REINFORCE, Actor–Critic (A2C), and DQN baselines reported in Section 4 are trained against the same weighted opponent pool used to train our modulation head, with an opponent re-sampled at the start of each episode (CFR $\times 3$, heuristic $\times 3$, six rule-based agents $\times 1$; see Section E). All three baselines share a compute budget of 200,000 episodes with learning rate 1×10^{-4}, batch size 32, and the Adam optimiser. Method-specific settings are summarised in Table 6.

Table 6: Baseline-specific hyperparameters.

Setting	**REINFORCE**	**Actor–Critic / DQN**
Discount γ	1.0	0.99
Entropy bonus coefficient	0.01	0.01 (A2C only)
Value-loss coefficient	—	0.5 (A2C only)
Replay buffer size	—	10,000 (DQN)
Target-network update	—	every 500 episodes (DQN)
Exploration	Boltzmann (policy)	ε-greedy, $1.0 \to 0.05$ over 200,000 episodes (DQN)

D Opponent Statistics

The opponent-statistics vector $o \in \mathbb{R}^7$ comprises six round-stratified behavioural rates and a confidence scalar. Each rate is maintained online by the learner during a 100-hand session and is reported as a Beta–Bernoulli posterior mean smoothed toward a pool-average prior.

Posterior-mean smoothing. For each rate, let k denote the count of qualifying events (e.g. folds) and n the count of qualifying contexts (e.g. all pre-flop actions). With prior strength $S = 20$ and pool-mean prior $\bar{p}$,

$$\hat{p} \;=\; \frac{k + \alpha}{n + \alpha + \beta}, \qquad \alpha = \bar{p} \cdot S,\ \beta = (1 - \bar{p}) \cdot S.$$

The pool means $\{\bar{p}\}$ are calibrated once, before training, by playing 500 hands against each pool opponent with a uniform-random learner (so that every action context is covered) and averaging the observed counts across opponents.

Feature definitions.

Pre-flop fold rate. Fraction of pre-flop actions in which the opponent folded. High values indicate a passive or tight pre-flop range.

Pre-flop raise rate. Fraction of pre-flop actions that were raises. High values indicate aggression or wide pre-flop ranges.

Flop raise rate. Fraction of flop actions that were raises. A spike relative to the pre-flop raise rate is a strong "opponent paired the board" signal.

Pre-flop fold-to-raise. Among pre-flop actions taken while facing a raise, the fraction that were folds. Distinguishes opponents who fold to pressure pre-flop from those who call down.

Flop fold-to-raise. The analogous rate for the flop round. Combined with the flop raise rate this characterises post-flop defence frequency.

Raise-after-raise rate. Among actions taken while facing a raise and with a re-raise legal, the fraction that were raises. A maniac-style aggressor scores near 1.0; passive opponents score near 0.0.

Confidence. $\rho = n_h/(n_h + S)$, where n_h is the number of hands observed against the current opponent within the session and $S = 20$. The scalar grows from 0 at the start of a session to ≈ 0.83 after 100 hands and allows the modulation head to discount the other six features during the early, statistically-noisy hands of a session.

E Agent Pool and Evaluation Protocol

E.1 Training Pool

The modulation head is trained against a fixed pool of eight opponents that are sampled per session under the weighting in Table 7. Such sampling ensures that roughly half of the training budget is spent playing against high-quality and challenging opponents (CFR and hand-craftet heuristic agent).The remaining six agents are simple style-based archetypes (tight-passive, loose-aggressive, etc.) that exhibit exploitable tendencies by construction.

Table 7: Training-pool opponents and per-session sampling weights. The expected per-session frequency for each agent is its weight divided by the total weight (14.0).

Opponent	Weight	Description
CFR	3.0	Tabular CFR$^+$; Nash-equilibrium baseline.
Heuristic	3.0	Hand-crafted expert-style strategy.
Tight-Passive	1.0	"Nit"; raises only made pairs, folds Js to pressure.
Tight-Aggressive	1.0	Raises premium hands; folds marginals under pressure.
Loose-Passive	1.0	Calling station; never folds pre-flop.
Loose-Aggressive	1.0	Bluff-raises Js pre-flop (40%) and on the flop (35%).
Maniac	1.0	Raises whenever legal; folds only J-high under pressure with raises capped.
Random	1.0	Uniform-random over legal actions.

E.2 Evaluation Pool

For the main results (Section 4 and Figure 2) we score each agent in a *fixed-gauntlet* round-robin against a 13-agent evaluation pool. Eight opponents are in-distribution — the same agents the modulation head observed during training — and five are out-of-distribution neural-network agents that the modulation head never trained against (Table 8).

Table 8: Evaluation pool. The out-of-distribution agents were trained independently for this study and never appeared in any modulation-head training session.

Opponent	Distribution	Notes
CFR	In-distribution	Tabular CFR$^+$ (Nash baseline).
Heuristic	In-distribution	Rule-based.
Tight-Passive	In-distribution	Rule-based archetype.
Tight-Aggressive	In-distribution	Rule-based archetype.
Loose-Passive	In-distribution	Rule-based archetype.
Loose-Aggressive	In-distribution	Rule-based archetype.
Maniac	In-distribution	Rule-based archetype.
Random	In-distribution	Uniform-random.
Opp. Encoder	Out-of-distribution	Frozen base + learned opponent encoder.
Adaptive Value	Out-of-distribution	Value net with 19-dim input (state + 4 hand-coded stats).
REINFORCE	Out-of-distribution	Policy gradient (200,000 pool-training episodes).
Actor–Critic	Out-of-distribution	A2C (200,000 pool-training episodes).
DQN	Out-of-distribution	Double Q-learning with replay and target network.

E.3 Match Protocol and Metrics

Each (study agent, pool agent) matchup is played for 10,000 hands with alternating seats: the learner sits at seat 0 for odd-indexed hands and seat 1 for even-indexed hands. Hands are grouped into

100-hand sessions, and the opponent-statistics tracker is reset at the session boundary so that cold-start behaviour is repeatedly probed.

Per-matchup metrics. Each matchup yields the learner's mean chips per hand together with a 95% confidence interval and a *cold* vs *warm* decomposition (hands 0–19 vs hands 20–99 within each session), used to isolate the contribution of opponent-statistics accumulation.

Pool-level metrics. For each study agent we report, across the 13 pool opponents (and separately across the in-distribution and OOD subsets):

$$\mu = \tfrac{1}{|P|} \sum_{p \in P} \bar{x}_p, \qquad \sigma = \mathrm{stddev}_{p \in P}(\bar{x}_p),$$

$$\text{robustness} = \mu - 1.5\,\sigma, \qquad \text{worst-case} = \min_{p \in P} \bar{x}_p,$$

where $\bar{x}_p$ is the mean chips per hand against opponent p. The robustness score penalises high variance across opponents, rewarding agents that perform consistently rather than those that exploit some opponents strongly while collapsing against others.

For comparability with the main text's tables, the per-100-hand session score is $100 \times \bar{x}_p$ (e.g. $+0.419$ chips per hand $= +41.9$ chips per 100-hand session).

Checkpoint selection. During training we evaluate every $10{,}000$ episodes against the in-distribution pool (500 rounds per opponent) and retain both the final checkpoint and the checkpoint that maximises the in-training robustness score, which is then used for the final fixed-gauntlet evaluation.